GRADUATION CAPSTONE | EXCELLENT RATING

AI-Integrated Security Operations Automation

Splunk + n8n + Threat Intelligence + OpenAI + Slack

A step-by-step technical case study

4	372/373	~9.3 sec	MITRE ATT&CK
validated detection scenarios	end-to-end alert deliveries	weighted average response latency	mapped, enriched, actionable triage

NGUYEN THAI HOANG

Cybersecurity | SIEM | SOAR | Detection Engineering | AI for SOC

PUBLIC PORTFOLIO EDITION * SANITIZED FOR PROFESSIONAL SHARING

02 PROJECT AT A GLANCE

A portfolio summary for recruiters, security leaders, and engineering teams

I designed and implemented a small-scale Security Operations Center (SOC) proof of concept that automates alert triage from endpoint telemetry to an analyst-ready Slack message.

4	372 / 373	~9.3 s	~93%
Detection scenarios	**Alert deliveries**	**Weighted latency**	**AI label accuracy**
Brute force, malware, SYN flood, CIC-IDS	~99.7% end-to-end workflow success	Log timestamp to Slack notification	Measured on the CIC-IDS classification test

What I personally built

Platform Engineering

- Provisioned the virtual lab and isolated network topology.
- Deployed Splunk Enterprise, Universal Forwarder, and n8n on Docker.
- Configured persistent storage, service startup, and permissions.

Security & Automation Engineering

- Collected targeted Windows security telemetry and created four SPL detection rules.
- Normalized heterogeneous alert payloads with JavaScript.
- Integrated OpenAI, AbuseIPDB, and dynamic Slack routing.

PUBLIC EDITION Credentials, API keys, webhook URLs, and sensitive implementation details have been removed or sanitized. All simulations were performed in an isolated lab.

03 THE PROBLEM I SET OUT TO SOLVE

Reduce repetitive Tier-1 triage while preserving context and actionable guidance

Traditional alert handling forces a junior analyst to switch between SIEM searches, threat-intelligence websites, MITRE ATT&CK references, and communication tools. My goal was to turn those disconnected steps into one automated, auditable workflow.

Project Objectives

- Collect endpoint logs continuously from Windows 10.
- Detect suspicious authentication, PowerShell, and network behavior.
- Enrich source indicators with external reputation data.
- Use AI to summarize, assign severity, map MITRE ATT&CK, and recommend response actions.
- Deliver a clear message to the correct Slack incident channel.

Success Criteria

- No manual intervention between alert trigger and notification.
- Consistent event schema across multiple log sources.
- Response latency measured from Splunk event time to Slack time.
- Repeatable tests with documented counts and timestamps.
- Clear separation between delivery success and AI classification accuracy.

Core engineering principle

ACTIONABLE OVER VERBOSE The AI output was constrained to four tasks: summarize the incident, enrich relevant indicators, assess severity with reasoning, and recommend concrete detection/prevention/response actions.

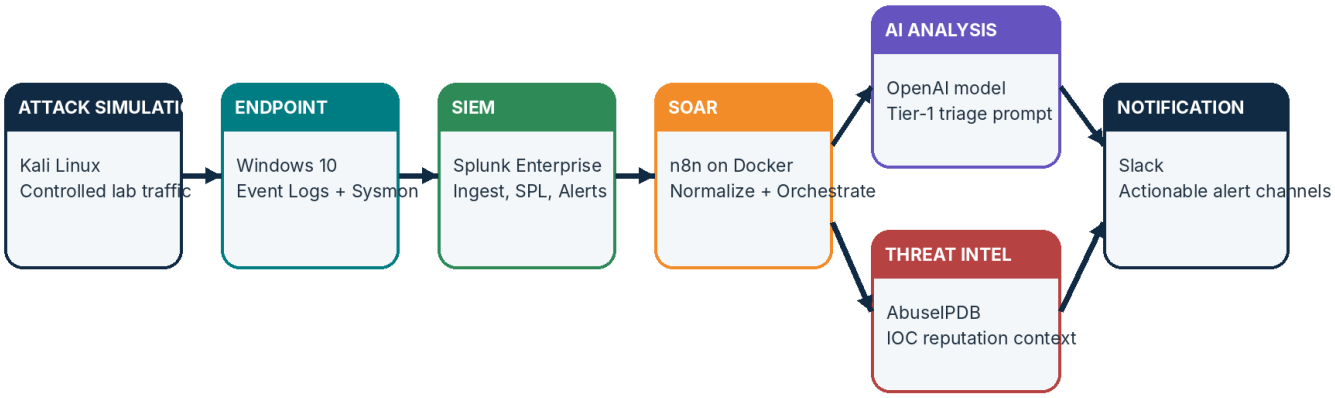
Project scope: a controlled proof of concept, not a production SOC. The design intentionally emphasizes transparency and measurable workflow behavior.

04 SYSTEM ARCHITECTURE

A modular pipeline with clearly separated detection, orchestration, analysis, and notification layers

End-to-End Architecture

From controlled attack simulation to enriched analyst notification



Primary event path: endpoint telemetry → detection → orchestration → AI/threat-intel enrichment → Slack

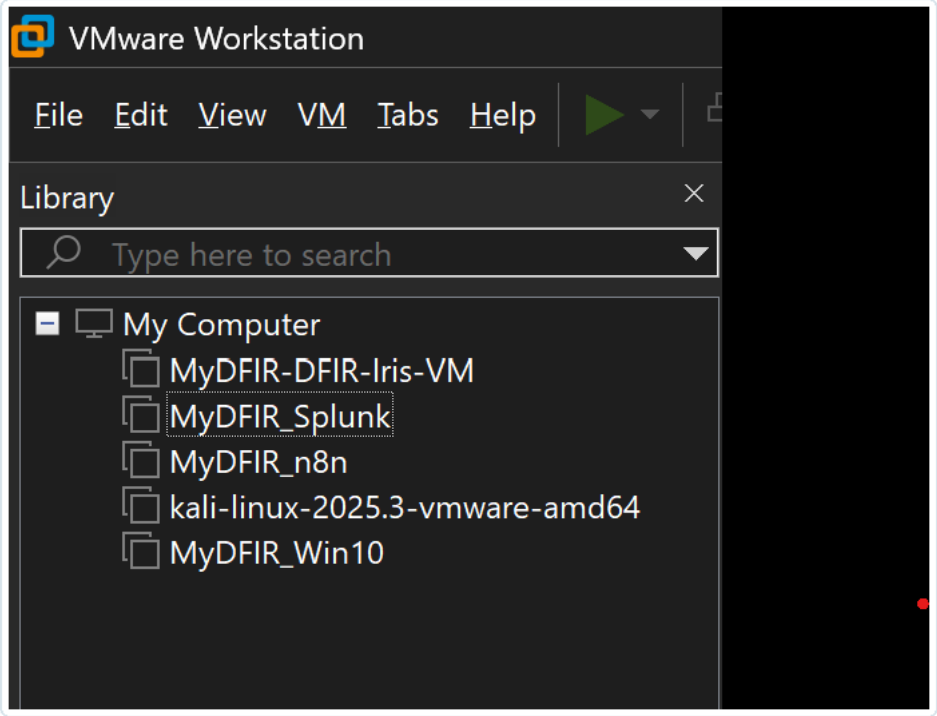
Data flow

Windows telemetry is forwarded to Splunk. SPL rules create structured alerts and call an n8n webhook. n8n normalizes fields, optionally enriches the source IP through AbuseIPDB, sends contextual data to the AI analyst prompt, and posts the result to a scenario-specific Slack channel.

> **DESIGN CHOICE** Splunk remains the detection authority; n8n handles orchestration; AI adds context and recommendations. This separation avoids treating the language model as the primary detector.

05 LAB TOPOLOGY AND ENVIRONMENT

Five virtual machines on an isolated VMware network



VMware Workstation inventory used for the project lab.

Role	Platform	Purpose
Victim endpoint	Windows 10 x64	Generate Windows Event Logs and host the Universal Forwarder
SIEM server	Ubuntu Server 24.04 LTS	Run Splunk Enterprise and store the project index
SOAR server	Ubuntu Server 24.04 LTS	Run n8n in Docker and orchestrate the workflow
Attacker / simulator	Kali Linux	Generate controlled authentication and network test traffic
Case-management VM	Ubuntu Server	Provisioned for future expansion; not part of the evaluated alert path

REPRODUCIBILITY I took a clean base snapshot before configuration so the environment could be reset and test runs could start from a known state.

06 BUILD PHASE 1 - ENDPOINT TELEMETRY

Collect only the Windows log channels needed for detection and investigation

Telemetry Sources

- Sysmon Operational - process, network, file, and IOC visibility.
- Windows Defender Operational - malware detections and security events.
- PowerShell Operational - script-block and fileless activity.
- Security - logons, failed authentication, privilege events, process creation.
- System and RDP Session logs - services, drivers, and remote-session context.

Forwarder Configuration

- Installed Splunk Universal Forwarder on Windows 10.
- Forwarded data to the Splunk receiving port (9997).
- Used a dedicated index: mydfir-project.
- Ran the forwarder as Local System to access protected event channels.
- Restarted the service and validated ingestion in Splunk.

```
[WinEventLog://Microsoft-Windows-Sysmon/Operational]
index = mydfir-project
disabled = false
renderXml = true
source = XmlWinEventLog:Microsoft-Windows-Sysmon/Operational

[WinEventLog://Microsoft-Windows-Windows Defender/Operational]
index = mydfir-project
disabled = false
source = Microsoft-Windows-Windows Defender/Operational
blacklist = 1151,1150,2000,1002,1001,1000

[WinEventLog://Microsoft-Windows-PowerShell/Operational]
index = mydfir-project
disabled = false
source = Microsoft-Windows-PowerShell/Operational
blacklist = 4100,4105,4106,40961,40962,53504

[WinEventLog://Application]
index = mydfir-project
disabled = false

[WinEventLog://Security]
index = mydfir-project
disabled = false

[WinEventLog://System]
index = mydfir-project
disabled = false

[WinEventLog://Microsoft-Windows-TerminalServices-LocalSessionManager/Operational]
index = mydfir-project
disabled = false
```

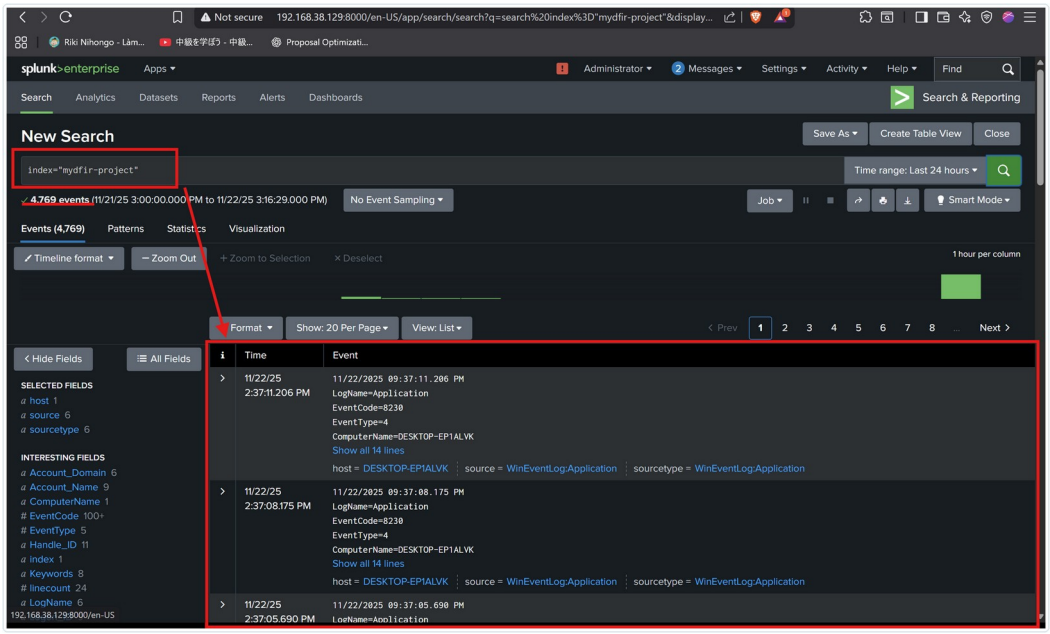
Sanitized inputs.conf excerpt defining the Windows event channels.

Representative configuration pattern

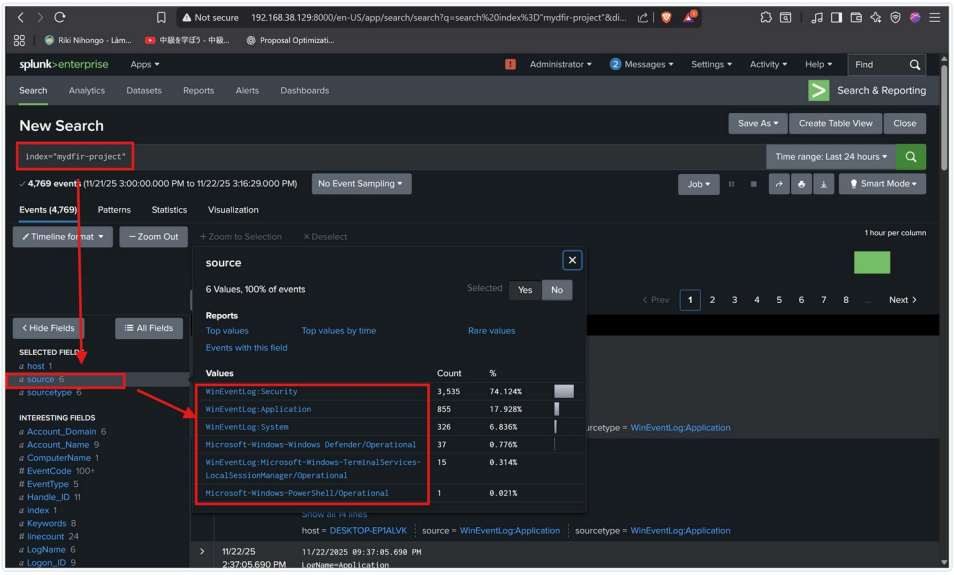
```
[WinEventLog://Security]
index = mydfir-project
disabled = false

[WinEventLog://Microsoft-Windows-PowerShell/Operational]
index = mydfir-project
disabled = false
```

07 BUILD PHASE 2 - SIEM AND DETECTION

Configure receiving, indexing, searches, and alert actions in Splunk



Validation query confirming endpoint logs in the project index.



Source review confirmed six of seven configured event channels were actively producing data.

What I Configured

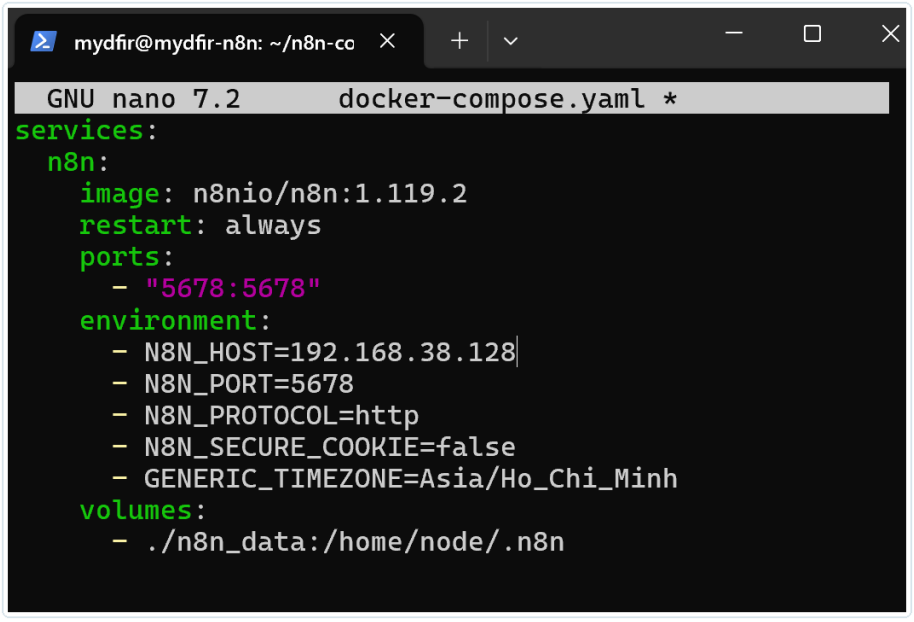
- Splunk Enterprise on Ubuntu Server.
- TCP receiving port 9997.
- Custom index mydfir-project.
- Microsoft security add-on for field normalization.
- Scheduled and real-time alerts with webhook actions.

How I Verified It

- Searched the project index for fresh Windows events.
- Reviewed source distribution to confirm expected channels.
- Generated known test events and compared endpoint/SIEM timestamps.
- Saved the search and attached an n8n webhook action.

08 BUILD PHASE 3 - DEPLOY N8N AS THE SOAR LAYER

Self-hosted workflow automation on Docker and Ubuntu Server



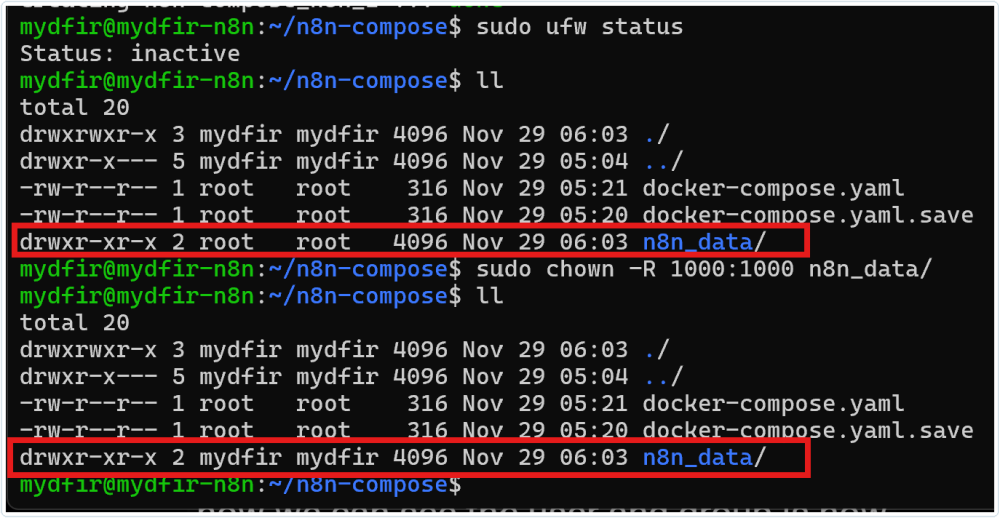
Docker Compose configuration for the self-hosted n8n service.

Sanitized deployment pattern

```
services:
  n8n:
    image: n8nio/n8n:<pinned-version>
    restart: always
    ports:
      - "5678:5678"
    volumes:
      - ./n8n_data:/home/node/.n8n
```

Issue solved: persistent-volume permissions

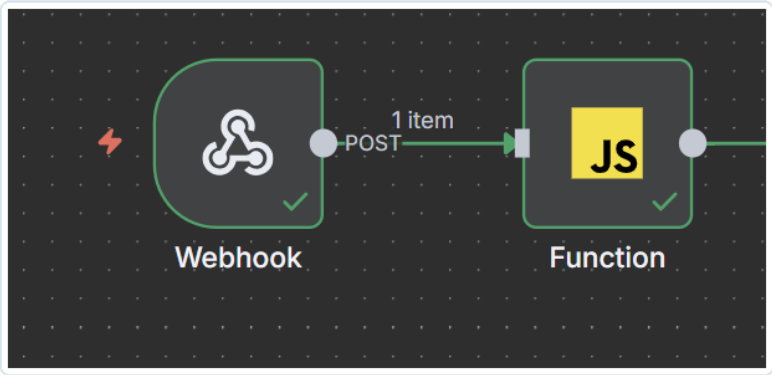
Docker created the host data directory as root, while the n8n container ran as UID 1000. This caused a permission-denied failure. I corrected ownership recursively and verified that n8n could persist workflows and credentials after restart.



Verification after correcting ownership of the n8n data directory.

09 BUILD PHASE 4 - NORMALIZE AND ROUTE ALERT DATA

Full JavaScript implementation used in the n8n Function node



Webhook entry point connected to the JavaScript Function node.

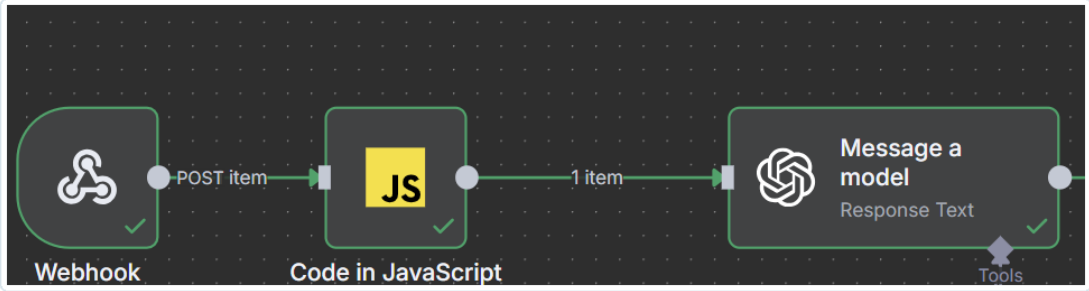
Complete normalization and dynamic-routing source code (comments translated into English; logic unchanged)

```
const body = items[0].json.body;
const result = body.result || {}; // Safe access

let extracted = {
  search_name: body.search_name || 'Unknown Alert',
  attack_type: result.attack_type || 'Unknown',
  severity: result.severity || 'Informational',
  ip_address: result.ip_address || result.Source_Address || result.SRC_IP || 'N/A',
  computer_name: result.ComputerName || result.Destination_Address || 'N/A',
  time: result._time || 'N/A',
  // Extract fields specific to each attack type
  attempts: result.attempts || 'N/A', // Brute force
  connection_count: result.connection_count || 'N/A', // DDoS
  message: result.Message || 'N/A', // PowerShell malware (full script when available)
  attack_type_cic: result.ATTACK_TYPE || 'N/A', // CIC-IDS legacy events
  raw_details: JSON.stringify(result, null, 2) // Retained for debugging/reference
};

const searchName = extracted.search_name || 'Unknown';
if (searchName.includes('SOC-AL-04')) {
  extracted.channel_name = '#alert-cic-ids';
} else if (searchName.includes('SOC-AL-01')) {
  extracted.channel_name = '#alert-brute-force';
} else if (searchName.includes('SOC-AL-02')) {
  extracted.channel_name = '#alert-malware';
} else if (searchName.includes('SOC-AL-03')) {
  extracted.channel_name = '#alert-ddos';
} else {
  extracted.channel_name = '#alert-general';
}

return [{ json: extracted }];
```

ENGINEERING PURPOSE The function applies safe defaults, coalesces inconsistent source fields into one schema, retains raw evidence for reference, and routes each SOC-AL rule to its dedicated Slack channel.

10 BUILD PHASE 5 - DESIGN THE AI ANALYST PROMPT

Prompt architecture and control logic for repeatable Tier-1 triage



Webhook -> JavaScript normalization -> AI message model.

Step	AI Task	Expected Output
1	Summarize	What happened, affected host/user, key evidence
2	Enrich	Use supplied threat-intelligence context only when relevant
3	Assess	Severity, MITRE ATT&CK technique, and explicit reasoning
4	Recommend	Detection, prevention, and response actions tailored to the alert

Dynamic severity logic

- Brute force: map to T1110; increase severity for high attempt counts or high-risk source reputation.
- Suspicious PowerShell: map to T1059.001; treat reverse-shell indicators as Critical.
- SYN flood: map to T1498; use connection volume and threat context to determine High/Critical.
- CIC-IDS simulation: preserve informational handling for benign or legacy dataset events.

WHY TWO PROMPTS The System prompt defines the analysis procedure and decision rules. The User prompt injects normalized event data with conditional fields, so the model receives scenario-relevant evidence without losing the raw log context.

11 BUILD PHASE 5A - FULL SYSTEM PROMPT

Verbatim prompt used to define the AI analyst role, severity logic, MITRE mapping, and response guidance

System role - complete implementation prompt

```
As a Tier 1 SOC analyst assistant. When provided with a security alert or incident details (including indicators of compromise, logs or metadata), perform the following steps without duplication or irrelevant analysis. Classify the alert based on the provided Attack Type and details (e.g., Event Codes, Ports) to tailor your response uniquely for each type, ensuring the Tier-1 analyst can immediately identify the attack (e.g., brute-force RDP, malware reverse shell, DDoS SYN flood, or legacy CIC-IDS event).

1. Summarize the alert - Provide a clear, concise summary of what triggered the alert, affected systems/users, and nature of activity. Tailor to alert type:
- For "Brute Force Attempt (Network/Auth Level)" (EventCode=4625 or 5156, Destination_Port=3389): Highlight failed login attempts (>15 in 5m), source IP, and RDP target.
- For "Malware Execution (Reverse Shell)" (EventCode=4104): Analyze suspicious PowerShell script (e.g., hidden window, TCPClient/WebClient/iex for reverse shell).
- For "SYN Flood DDoS" (EventCode=5156, connection_count >500 in 10s): Note high connection volume from source IP to non-Splunk ports (!=9997).
- For CIC-IDS legacy events (EVENT=CIC_IDS_SIM): Reference ATTACK_TYPE, SRC_IP/DST_IP, as informational simulation.

2. Enrich with threat intelligence - Correlate IOCs (IPs, domains, hashes) with known sources. Use 'AbuseIPDB-Enrichment' data (provided in context) only if IP is available and relevant. Skip for non-IP alerts:
- Brute-force/DDoS/CIC-IDS: Enrich source IP if present, highlight associations with malware/threat actors and the Abuse Confidence Score.
- Malware (EventCode=4104): Skip IP enrichment; focus on script patterns instead.

3. Assess severity (Dynamic Logic) - Map to MITRE ATT&CK tactics/techniques without overlap. Provide/adjust severity (Low/Medium/High/Critical) with reasoning.
CALCULATION LOGIC: Final_Severity = MAX(Base_Severity, Threat_Intel_Modifier)
- Brute-force: T1110 (Credential Access). Start with HIGH if attempts >15.
  * UPGRADE to CRITICAL if Abuse Confidence Score > 80% or IP is a Tor Exit Node.
- Malware: T1059 (Command and Scripting Interpreter). Start with CRITICAL if reverse shell indicators are found.
- DDoS: T1498 (Network Denial of Service). Start with HIGH if volume is high.
  * UPGRADE to CRITICAL if multiple sources are identified or Abuse Score is extreme.
- CIC-IDS: Informational, map based on ATTACK_TYPE (e.g., T1498 for DoS simulations).
  * DOWNGRADE to Informational if the Label is BENIGN or IP is in a known Whitelist.

REASONING: Explain clearly why you chose or adjusted the severity level based on the logic above.

4. Recommend next actions - Base recommendations directly on MITRE ATT&CK mitigations for the mapped technique, tailoring to the alert without generic advice. Include detection, prevention, and response steps:
- For T1110 (Brute Force):
  * Detection: Monitor failed logons/high attempts.
  * Prevention: Implement account lockout after failed attempts, use MFA, follow NIST password guidelines.
  * Response: Reset compromised accounts, block IP via firewall.
- For T1059.001 (Malware, EventCode=4104):
  * Detection: Monitor behavioral chains (unusual processes/network from PowerShell).
  * Prevention: Enforce code signing (signed scripts only), disable PowerShell if unnecessary, use Constrained Language mode.
  * Response: Isolate host, terminate processes, collect logs (command history/Base64), remove persistence (registry keys).
- For T1498 (DDoS):
  * Detection: Monitor large outbound traffic/flood tools.
  * Prevention: Use ISP/CDN for traffic filtering, block targeted ports/protocols.
  * Response: Block source IPs, implement disaster recovery plan.
- For CIC-IDS (based on ATTACK_TYPE): Use general MITRE mitigations (e.g., for DoS simulations, same as T1498), but keep informational with no urgent actions.

Format output clearly in Markdown for Slack.
```

12 BUILD PHASE 5B - FULL USER PROMPT

Verbatim n8n template used to inject normalized alert fields and scenario-specific evidence

User role - complete dynamic alert template

```
Alert: {{ $json.search_name }}
Type: {{ $json.attack_type }}{% if $json.attack_type_cic != 'N/A' %} (CIC-IDS: {{ $json.attack_type_cic }}){% endif %}
Initial Severity: {{ $json.severity }}
IP Address: {{ $json.ip_address }} (enrich if applicable)
Computer/Host: {{ $json.computer_name }}
Time: {{ $json.time }}

Key Details:
{% if $json.attempts != 'N/A' %}Failed Attempts (Brute Force): {{ $json.attempts }}{% endif %}
{% if $json.connection_count != 'N/A' %}Connection Count (DDoS): {{ $json.connection_count }}{% endif %}
{% if $json.message != 'N/A' %}PowerShell Script/Message (Malware): {{ $json.message }}{% endif %}
Raw Log for Reference: {{ $json.raw_details }}
```

Prompt-engineering decisions implemented

Conditional Context Injection

- Only scenario-relevant fields are displayed when their value is not N/A.
- Brute-force, DDoS, malware, and CIC-IDS evidence use one reusable template.
- The complete normalized result remains available through raw_details.

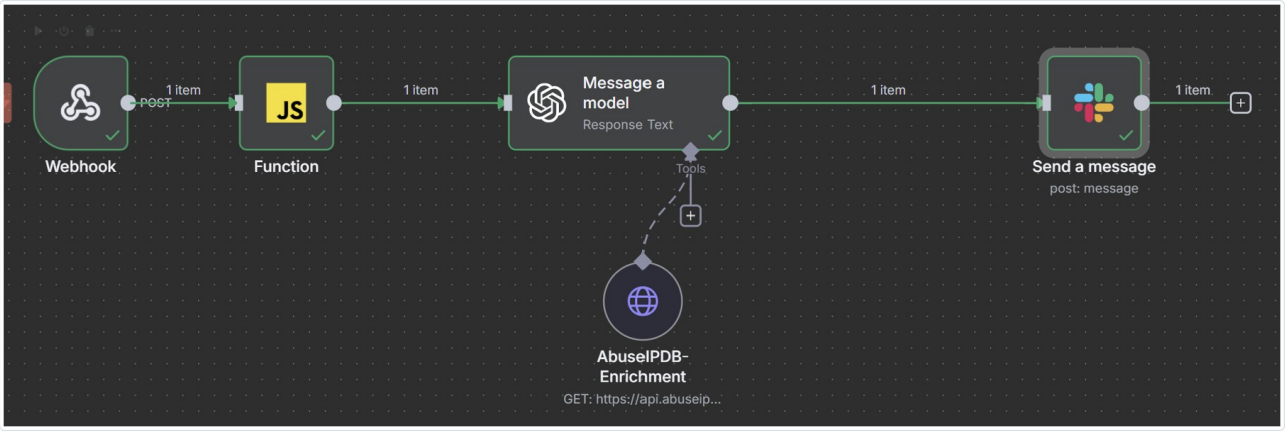
Model Control and Output

- The System role constrains the model to four ordered analyst tasks.
- Severity upgrades/downgrades follow explicit scenario and threat-intelligence rules.
- The response is formatted in Markdown for direct delivery to Slack.

IMPLEMENTATION DETAIL The prompt is downstream of the Function node, so every placeholder is populated from the normalized schema shown in Build Phase 4.

13 BUILD PHASE 6 - THREAT INTELLIGENCE AND ANALYST NOTIFICATION

Add external context and deliver the result to the correct incident channel



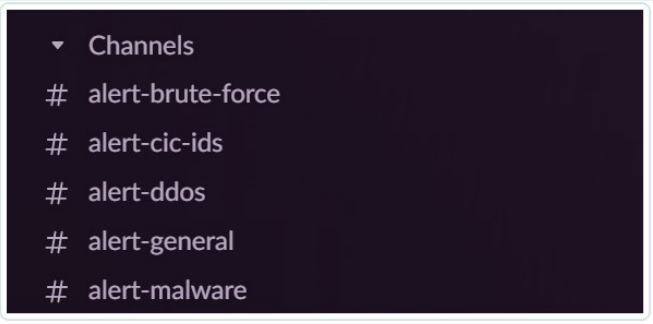
Final orchestration: Webhook, normalization, AI analysis, AbuseIPDB tool, and Slack notification.

Threat-Intelligence Enrichment

- Extract the source IP after normalization.
- Query AbuseIPDB for provider and confidence context.
- Pass the returned context to the AI prompt.
- Skip IP enrichment when it is not relevant, such as script-only malware events.

Slack Delivery

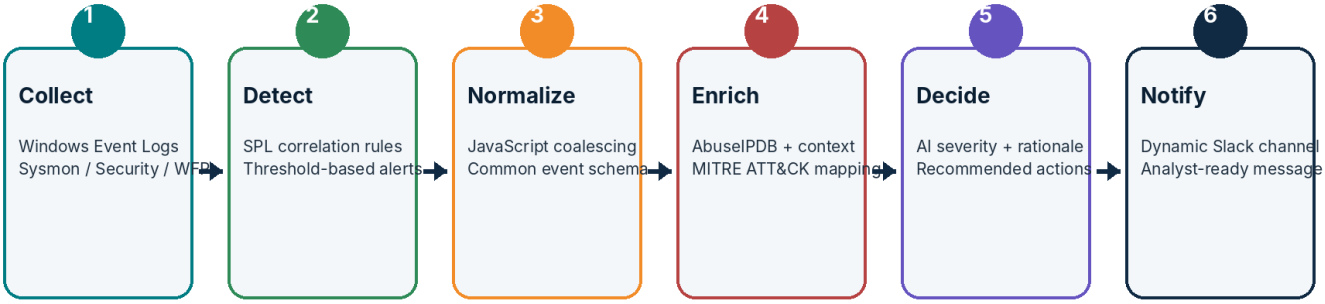
- One Slack node with a dynamic channel expression.
- Separate channels reduce alert fatigue and ownership ambiguity.
- Message includes summary, severity, MITRE mapping, rationale, and recommended actions.
- Fallback channel prevents unknown alert types from being dropped.

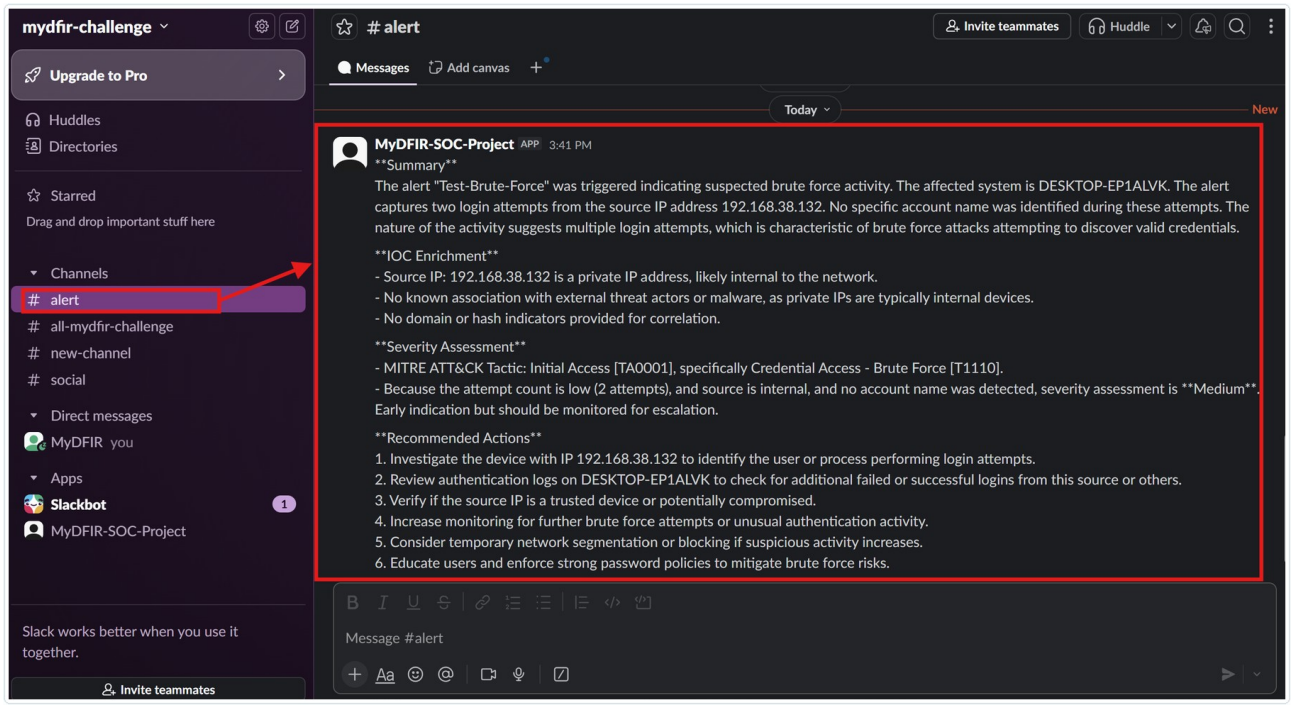


Incident-specific Slack channel structure.

14 END-TO-END WORKFLOW VERIFICATION

Validate every handoff before running large test batches





Example analyst-ready Slack output containing incident context and response recommendations.

VERIFICATION CHECKPOINTS I tested log arrival, detection search results, webhook payload reception, normalized fields, AI output, Slack app permissions, channel membership, and final message formatting independently before full-scale testing.

15 DETECTION ENGINEERING OVERVIEW

Four detection rules mapped to distinct telemetry and response logic

Rule	Scenario	Primary Evidence	Logic	MITRE
SOC-AL-01	RDP brute force	Event 4625 / 5156, port 3389	>15 attempts in 5 minutes	T1110
SOC-AL-02	Suspicious PowerShell	Event 4104 script-block content	Hidden PowerShell + network/exec indicators	T1059.001
SOC-AL-03	SYN flood	Event 5156 WFP connections	>500 connections per 10-second bucket	T1498
SOC-AL-04	CIC-IDS-2017 events	Normalized dataset fields	ATTACK_TYPE-driven analysis and routing	Technique varies

Representative SPL patterns

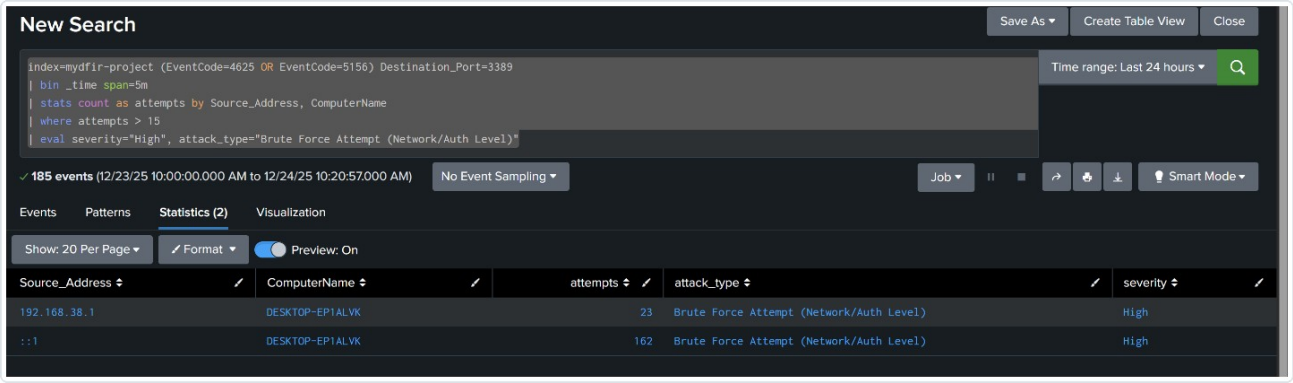
```
# Brute force
index=mydfir-project (EventCode=4625 OR EventCode=5156) Destination_Port=3389
| bin _time span=5m
| stats count as attempts by Source_Address, ComputerName
| where attempts > 15

# SYN flood
index=mydfir-project EventCode=5156 Destination_Port!=9997
| bucket _time span=10s
| stats count as connection_count by _time, Source_Address, Destination_Address, Destination_Port
| where connection_count > 500
```

RULE PHILOSOPHY Each rule produces a compact, structured result for automation instead of forwarding a large raw event set downstream.

16 SCENARIO 1 - RDP BRUTE-FORCE DETECTION

Controlled repeated login failures to validate authentication and network-layer visibility

Test objective: confirm that the endpoint, SIEM, detection rule, n8n workflow, AI analysis, and Slack delivery operate as one closed loop.



SOC-AL-01 SPL detection rule and alert configuration evidence.

Detection Logic

- Correlate failed logons (4625) and RDP connection events (5156).
- Restrict the traffic to destination port 3389.
- Aggregate events in five-minute windows.
- Trigger when attempts exceed the lab threshold.

AI Triage Output

- Identify likely credential-access activity.
- Map the event to MITRE ATT&CK T1110.
- Review source and target context.
- Recommend investigation, blocking, account reset, and continued monitoring.

LAB SAFETY The public portfolio intentionally omits attack scripts, credential lists, tunnel endpoints, and evasion instructions. The scenario is presented only as defensive validation.

17

SCENARIO 1 RESULTS

100 test cycles with complete alert delivery

100 / 100
Detected and delivered
Complete SIEM-to-Slack workflow success

8.0 s
Average latency
Five randomly sampled event/Slack pairs

0%
Observed false positives
Within the controlled lab test set

T1110
MITRE mapping
Credential Access - Brute Force

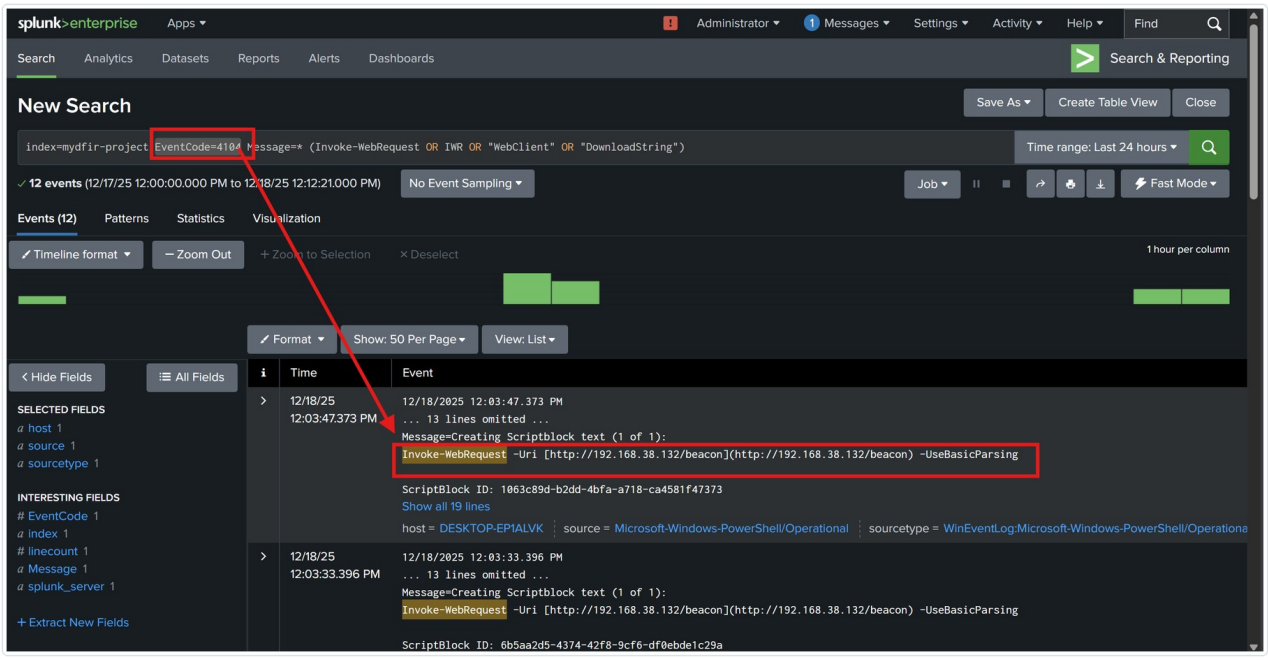
Sample	Splunk Time	Slack Time	Latency
1	21:00:17	21:00:30	13 s
2	21:01:33	21:01:40	7 s
3	21:02:36	21:02:44	8 s
4	21:03:49	21:03:56	7 s
5	21:05:05	21:05:10	5 s
Average			8.0 s

Result interpretation

- The closed-loop workflow required no manual analyst action after the alert condition was met.
- The AI consistently identified credential-access behavior and returned clear containment and investigation steps.
- The test measured delivery behavior in a controlled environment; it does not represent enterprise false-positive rates.

18 SCENARIO 2 - SUSPICIOUS POWERSHELL DETECTION

Detect script-block indicators associated with hidden command execution and reverse-shell behavior



Splunk captured the PowerShell 4104 event used by the malware detection rule.

SOC-AL-02 detection logic

```
index=mydfir-project EventCode=4104 Message=*
("Start-Process" AND ("powershell.exe" OR "$PSHOME") AND "-WindowStyle Hidden")
AND ("TCPClient" OR "System.Net.Sockets" OR "WebClient" OR
     "Invoke-WebRequest" OR "Invoke-RestMethod" OR "iex")
| eval severity="Critical", attack_type="Malware Execution (Reverse Shell)"
```

Why This Rule Is Targeted

- Requires hidden PowerShell execution plus network/command indicators.
- Uses script-block logging rather than relying only on file signatures.
- Returns the script message and host context for investigation.

Expected Analyst Actions

- Isolate the affected endpoint.
- Terminate suspicious processes and preserve evidence.
- Collect command history, encoded payloads, persistence artifacts, and related network logs.
- Escalate to Tier 2 / incident response.

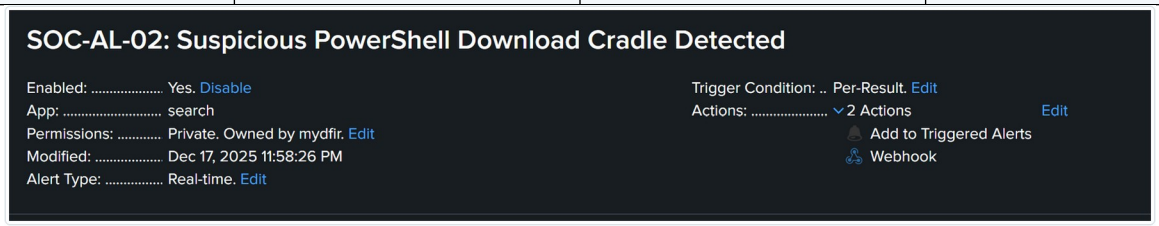
19

SCENARIO 2 RESULTS

Five controlled executions detected and analyzed

5 / 5	5 / 5	7.8 s	Critical
Splunk detections	**Slack notifications**	**Average latency**	**Assigned severity**
100% within the controlled test	No alert delivery loss	Event timestamp to Slack message	Reverse-shell indicators detected

Sample	Splunk Time	Slack Time	Latency
1	18:14:10	18:14:18	8 s
2	18:24:21	18:24:28	7 s
3	18:31:28	18:31:37	9 s
4	19:35:22	19:35:30	8 s
5	19:36:22	19:36:29	7 s
Average			7.8 s

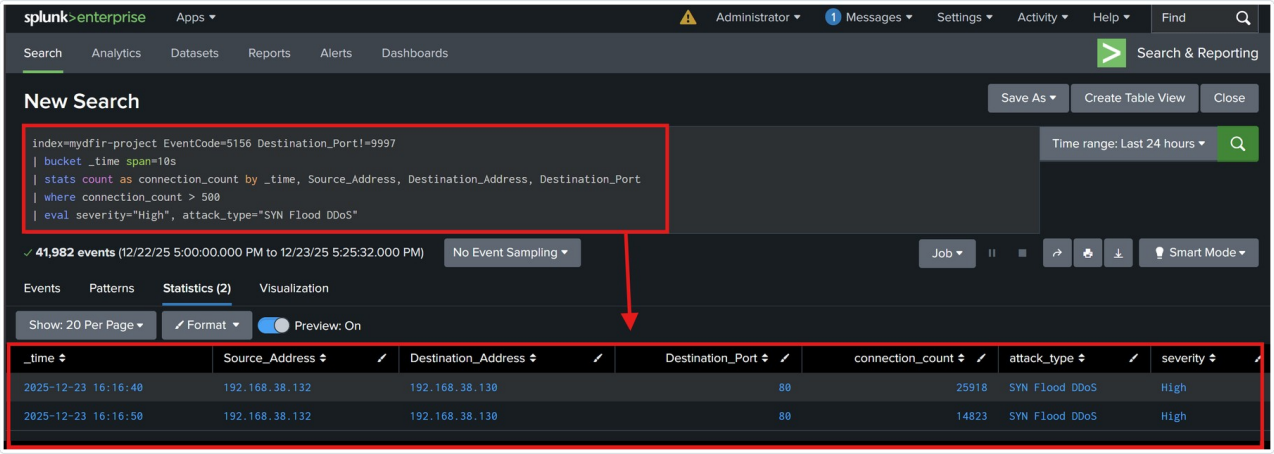


Saved alert evidence for SOC-AL-02: Suspicious PowerShell Download Cradle Detected.

KEY LEARNING Behavioral telemetry provided richer evidence than a file-only alert: the AI could explain the suspicious script pattern and recommend forensic collection steps.

20 SCENARIO 3 - SYN-FLOOD DETECTION

Measure high-volume network-event handling and notification stability



SOC-AL-03 SPL query detecting high connection counts in 10-second buckets.

Detection Method

- Enable Windows Filtering Platform connection auditing.
- Collect Event 5156 from the victim endpoint.
- Exclude the Splunk forwarding port to reduce self-generated noise.
- Aggregate by source, destination, port, and 10-second time bucket.
- Trigger above 500 connections per bucket.

Why This Test Matters

- Generates a burst of alerts instead of a single isolated event.
- Tests the workflow under sustained input pressure.
- Reveals queueing, API latency, and Slack delivery behavior.
- Validates severity reasoning against changing connection counts.

INTERPRETATION The lab measured detection and orchestration under load; it was not designed to demonstrate the availability impact of a real distributed denial-of-service attack.

21 SCENARIO 3 RESULTS

68 consecutive triggers processed during a 15-second test

68 / 68	8.8 s	~4.5/s	T1498
Alert deliveries	**Average latency**	**Peak alert rate**	**MITRE mapping**
Complete trigger-to-Slack success	Across beginning and ending samples	Observed workflow throughput	Network Denial of Service

Phase	Samples	Average Latency	Observation
Beginning	5	10.0 s	Initial API/workflow warm-up
Ending	6	7.83 s	No latency accumulation at the end of the burst
Overall	68 triggers	~8.8 s	Stable notification flow under sustained input

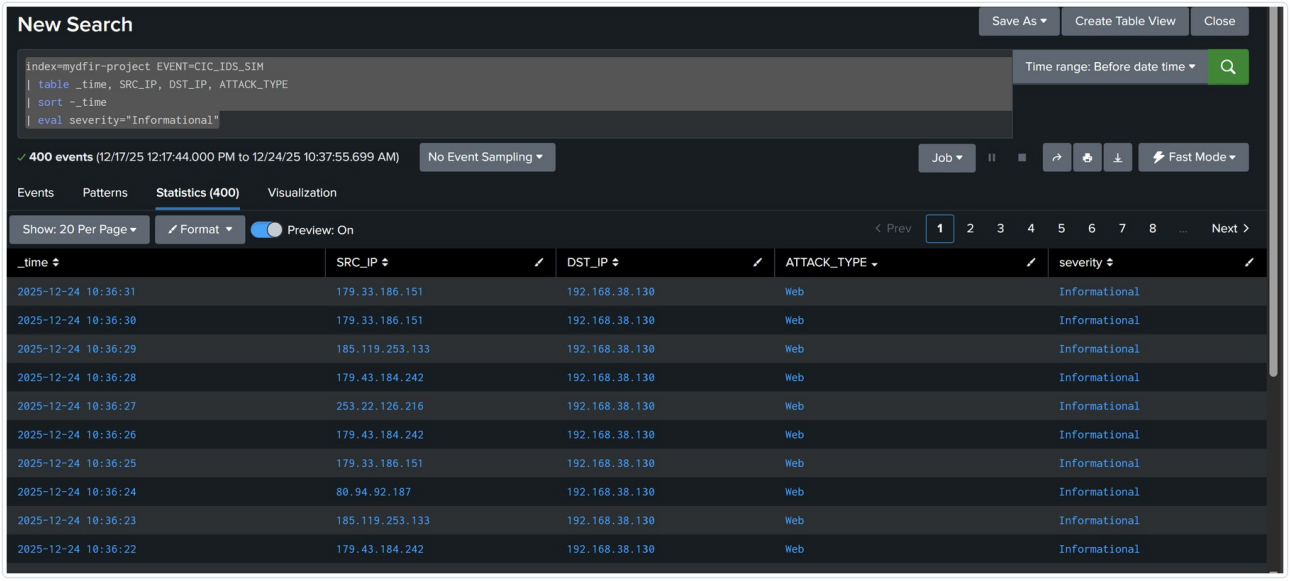
Engineering observations

- Latency did not increase as the alert burst progressed; ending samples were faster than beginning samples.
- Slack displayed near-simultaneous messages, indicating the workflow could complete multiple analyses without strict serial blocking.
- The result supports the architecture as a strong proof of concept, while production deployment would still require queues, retry policies, and rate-limit controls.

22 SCENARIO 4 - CIC-IDS-2017 DATASET VALIDATION

Test broader attack labels and sustained one-event-per-second ingestion

The final test used 200 benign and attack samples from the CIC-IDS-2017 dataset to evaluate data integrity, workflow delivery, and AI label classification beyond the three handcrafted scenarios.



SOC-AL-04 search and alert evidence for normalized CIC-IDS events.

SOC-AL-04 dataset query

```
index=mydfir-project EVENT=CIC_IDS_SIM
| table _time, SRC_IP, DST_IP, ATTACK_TYPE
| sort -_time
| eval severity="Informational", attack_type=ATTACK_TYPE
```

Test Design

- 200 total samples: attack and benign.
- Ingest rate: one event per second.
- Splunk used as the data-integrity checkpoint.
- Slack used as the end-to-end delivery checkpoint.

Evaluation Dimensions

- Did Splunk ingest every event?
- Did the SOAR workflow deliver every analyzed alert?
- How long did delivery take?
- Did the AI assign the correct attack label?

23 SCENARIO 4 RESULTS

High delivery reliability with one observed API/workflow miss

200 / 200	199 / 200	12.7 s	~93%
SIEM ingestion	**Slack delivery**	**Average latency**	**AI label accuracy**
No data loss at the Splunk layer	99.5% end-to-end workflow success	Ten timestamp pairs reviewed	Separate from delivery reliability

Metric	Result	Meaning
SIEM data integrity	100%	Universal Forwarder and Splunk received all test records
SOAR delivery reliability	99.5%	One of 200 AI/Slack notifications was not delivered
Average response latency	12.7 s	Slightly slower under continuous one-event-per-second load
AI classification accuracy	~93%	Model label accuracy on the dataset test

HONEST RESULT REPORTING The missed message was attributed in the project log to transient API throttling or service limits. I report 99.5% as workflow delivery reliability, not as model classification accuracy.

This distinction is important: SIEM ingestion, orchestration reliability, and model accuracy are different metrics and should be monitored separately in production.

24 EVALUATION METHODOLOGY

Repeatable test preparation and timestamp-based latency measurement

$$MTTR = T_{Slack} - T_{Splunk}$$

Latency formula used throughout the project.

Response latency was calculated as the Slack message timestamp minus the corresponding Splunk event timestamp. This avoids subjective stopwatch measurement and creates an auditable trail.

Step	Method
1. Clean start	Clear or isolate prior test events before each batch.
2. Generate controlled event	Run one defined scenario with known timing and volume.
3. Verify SIEM count	Confirm the expected number of records/triggers in Splunk.
4. Verify Slack count	Confirm the corresponding notification count in the target channel.
5. Sample timestamps	Pair Splunk _time with Slack message time and calculate the delta.
6. Separate metrics	Report ingestion, delivery, latency, and AI classification independently.

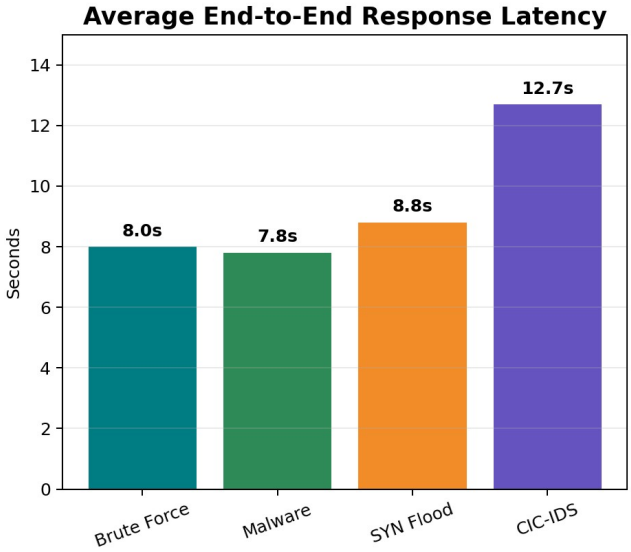
MEASUREMENT LIMITATION The latency values primarily measure automated processing after an event is available to the alert workflow. Scheduled-search cadence can add additional detection delay depending on configuration.

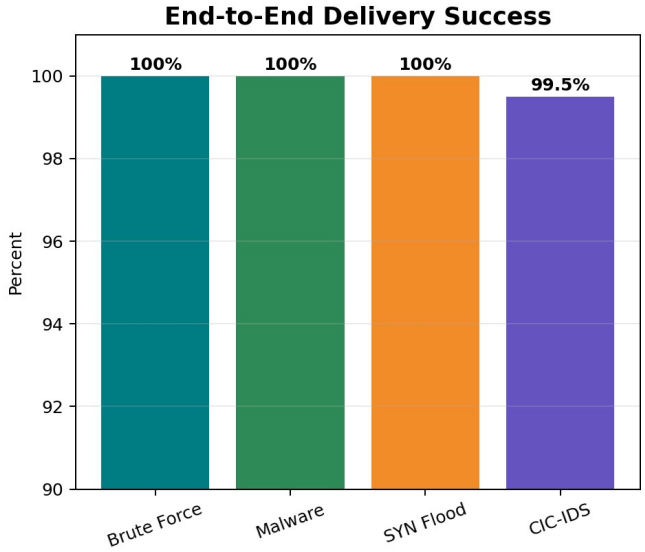
25

OVERALL RESULTS

Four scenarios, 373 total test events or alert triggers

Validated Lab Results





Scenario	Total	Delivered	Success	Avg. Latency
RDP brute force	100	100	100%	8.0 s
Suspicious PowerShell	5	5	100%	7.8 s
SYN flood triggers	68	68	100%	8.8 s
CIC-IDS events	200	199	99.5%	12.7 s
Weighted total	373	372	~99.7%	~9.3 s

BOTTOM LINE The project demonstrated a measurable closed-loop SOC workflow: targeted endpoint telemetry, SPL-based detection, automated enrichment and AI triage, and fast analyst notification.

26 ENGINEERING DECISIONS AND PROBLEMS SOLVED

The strongest evidence of hands-on work was in the integration details

Challenge	What I Did	Result
Noisy Windows telemetry	Selected specific event channels and excluded low-value Defender/PowerShell events	Lower ingestion noise and clearer alert context
Container permission failure	Aligned n8n data-directory ownership with container UID/GID	Persistent workflow storage became stable
Different source-field names	Implemented JavaScript coalescing and safe defaults	One common schema across Windows, WFP, and dataset events
Alert fatigue	Created incident-specific Slack channels with dynamic routing	Clear ownership and less channel noise
Context-poor alerts	Added AbuseIPDB enrichment before AI analysis	More specific severity reasoning and recommendations
High-volume input	Measured beginning/end latency during a 68-trigger burst	Confirmed no observed latency accumulation in the lab
API/workflow miss	Separated SIEM integrity, delivery reliability, and model accuracy	Transparent reporting of the 199/200 CIC-IDS result

WHAT THIS SHOWS Beyond installing tools, I debugged permissions, standardized data contracts, designed routing logic, tuned detection queries, and built a repeatable test-and-measurement process.

27

LIMITATIONS AND PRODUCTION ROADMAP

How I would evolve the proof of concept for an enterprise environment

From Proof of Concept to Production

Security

TLS everywhere, secrets manager, RBAC, API key rotation

Reliability

Queue, retries, rate-limit handling, dead-letter workflow

Scale

Multiple endpoints, distributed forwarders, HA Splunk/n8n

Detection Quality

Baselines, false-positive tuning, rule versioning, test harness

Governance

Human approval gates, audit logs, prompt/model evaluation

Case Management

Create incidents automatically and track analyst decisions

Current limitations

- Single-host virtual lab with no high availability or disaster recovery.
- External AI and threat-intelligence APIs introduce latency, availability, privacy, and rate-limit dependencies.
- Thresholds were tuned for controlled scenarios; enterprise baselines and false-positive testing are still required.
- The project measured automated response latency, not full incident containment or business-impact recovery time.
- Human approval gates should be added before any automated blocking or endpoint isolation action.

28 SKILLS DEMONSTRATED AND FINAL TAKEAWAYS

A defensive engineering project spanning infrastructure, detection, automation, AI, and measurement

Technical Skills

- Splunk Enterprise, SPL, Universal Forwarder, Windows Event Logs, Sysmon.
- n8n workflow automation, webhooks, JavaScript data normalization, Docker.
- OpenAI prompt design, structured triage, AbuseIPDB enrichment, Slack API.
- Detection engineering for authentication, PowerShell, and network behavior.
- Test design, timestamp analysis, latency measurement, and transparent metrics.

Professional Practices

- Modular architecture and clear component responsibilities.
- Evidence-driven validation at each integration boundary.
- Sanitization of credentials and sensitive configuration for public sharing.
- Separation of detector output, workflow reliability, and model accuracy.
- Ethical testing within an isolated and authorized environment.

Final takeaway

This capstone shows how I approach cybersecurity engineering: define a measurable problem, build the pipeline end to end, debug real integration failures, validate with repeatable tests, and communicate results without overstating what the data proves.

PORTFOLIO NOTE The original implementation journal contains 129 pages of screenshots, commands, configurations, and test evidence. This edition reorganizes that work into a concise US-English technical case study for LinkedIn and professional review.

Nguyen Thai Hoang | Cybersecurity * SIEM * SOAR * Detection Engineering * AI for Security Operations